

THE REEF

Factorio Space Age Mod — Plan of Work & Developer Reference

A salvage-yard space location focused on platform upgrades and Dilithium technology

Mod Concept Overview

The Reef is a new space location destination for Factorio: Space Age — analogous to the Shattered Planet but with its own identity. Rather than a barren rock, The Reef is a vast graveyard of alien starships drifting in deep space. As players approach and arrive, asteroids gradually give way to floating chunks of starship scrap that can be harvested, crushed, and recycled for resources.

The mod's content is structured around two pillars:

- A unique resource pipeline: Starship Scrap → raw materials via Crushers and Recyclers, plus rare Dilithium Crystals powering a unique tech tree.
- A suite of space platform enhancements: new generators, logistics structures, and thrusters that make operating in deep space significantly more capable.

 **Tip:** This document is designed to be used alongside Claude (desktop). Each phase is self-contained — you can hand Claude this doc and say 'let's start Phase 0' and it will have all the context it needs.

Developer Environment Setup

Before writing any Lua, set up a proper development environment. The investment pays off immediately through autocomplete, type-checking, in-game debugging, and dramatically longer Claude Code sessions.

Primary Tool: Claude Code in VS Code

Claude Code running inside VS Code is the recommended setup for this project. It gives Claude full codebase awareness, inline diff review, checkpoints to rewind bad edits, and direct access to the same files Factorio loads — no context-switching between windows.

- Install Claude Code: open VS Code, press Cmd+Shift+X (or Ctrl+Shift+X), search 'Claude Code', and install the Anthropic extension
- Sign in with your Claude account inside the panel that opens
- Default model: Claude Sonnet 4.6. Switch with /model opus for hard architectural problems, then back to sonnet for routine work
- The extension and the CLI share conversation history. Run `claude --resume` in the VS Code integrated terminal to continue a panel conversation in the CLI if you need a CLI-only feature

 **Tip:** Create a CLAUDE.md file in the mod's root folder on day one. Claude Code loads it automatically at the start of every session — put the mod concept, current phase, key design decisions, and API version there. This is your persistent memory across sessions.

Required: RTK — Rust Token Killer

RTK is a CLI proxy that compresses shell command output before it reaches Claude Code's context window. It installs as a hook into Claude Code's settings.json and runs transparently — you don't change how you work, commands are just rewritten automatically. Measured savings: 60–90% token reduction on common dev commands (git status: 80%, find: 78%). This directly extends your usable session length and reduces API costs.

 **Tip:** RTK is a system tool, not a skill file. It only works in Claude Code (not in chat). Install it once globally and it applies to all projects automatically.

Install instructions (macOS/Linux):

- Homebrew (recommended): `brew install rtk-ai/tap/rtk`
- Shell script: `curl -fsSL https://raw.githubusercontent.com/rtk-ai/rtk/master/install.sh | sh`
- From source: `cargo install --git https://github.com/rtk-ai/rtk`
- Windows: download the binary from GitHub Releases and place `rtk.exe` in your PATH. For best results use WSL.

After installing, verify and activate:

- Verify: `rtk gain` — must show token savings stats. If it fails, you installed the wrong 'rtk' package (there is an unrelated 'Rust Type Kit' on crates.io). Uninstall and install from the GitHub URL above.
- Activate for Claude Code: `rtk init -g` — installs the PreToolUse hook into Claude Code's settings.json
- Restart Claude Code after running init
- Check savings over time: `rtk gain --history`

Required: VS Code + Factorio Modding Toolkit (FMTK)

FMTK is the official community tool for Factorio mod development. Install it from the VS Code marketplace as the extension by justarandomgeek (search 'Factorio Modding Tool Kit'). It provides:

- Live debugging of Lua code running inside Factorio
- Auto-generated type definitions for the entire Factorio API (from runtime-api.json and prototype-api.json)
- Integration with the sumneko Lua language server for autocomplete and type checking
- Changelog and locale file syntax support

 **Tip:** After installing FMTK, click the Factorio version selector in the VS Code status bar. FMTK will automatically install the Lua type definitions for your version. This single step unlocks full API autocomplete.

Official repo: github.com/justarandomgeek/vscode-factoriomod-debug

Required: sumneko Lua Language Server

Install the 'Lua' extension by sumneko from the VS Code marketplace. FMTK will configure it automatically once you've selected your Factorio version. This gives you real-time error highlighting and cross-mod require path resolution.

Recommended: Lua Skill (mindrally/lua)

Install the mindrally/lua skill from the explainx.ai registry before starting Phase 0. It gives Claude solid general Lua fundamentals: table/metatable patterns, pcall/xpcall error handling, module organization with require(), local variable performance patterns, and game-loop design. This handles general Lua idioms; the Factorio modding skill we build during development handles the Factorio-specific API layer on top.

- Install via the Claude desktop app: Customize > Skills > Browse, then search for 'lua' by mindrally
- Or via CLI: `claude install-skill https://explainx.ai/skills/mindrally/skills/lua`
- Install before beginning Phase 0 so it is active for all subsequent sessions

Recommended: Workspace Layout

Open your Factorio mods directory as the VS Code workspace root, then check out The Reef mod's git repo directly into that folder. This lets FMTK auto-detect mod-list.json and configure the debugger with minimal setup.

- Mods directory (Windows): `%APPDATA%\Factorio\mods\`
- Mods directory (Mac): `~/Library/Application Support/factorio/mods/`
- Mods directory (Linux): `~/.factorio/mods/`

Recommended: Additional VS Code Extensions

- indent-rainbow — Colors indentation levels, very helpful since Lua uses fewer brackets than most languages
- Git Graph — Visual git log for more complex git operations
- Error Lens — Shows errors and warnings inline in the editor

Modding Libraries & Resources

Core Libraries (Add as Mod Dependencies)

Tool / Resource	Type	Purpose
flib	<i>Lua Library</i>	The standard go-to utility library for Factorio mods. Provides GUI helpers, event handler wrappers, table/math/string utilities, and more. Actively maintained and used by most major mods. Require with <code>require("__flib__.module-name")</code> .

PlanetsLib	<i>Planet Helper</i>	Community library specifically for building new planets, moons, and space locations. Provides orbit graphics generation, rocket-part recipe assignment per planet, cargo-drop tech helpers, and compatibility with other planet mods. Strongly recommended for The Reef.
stdlib	<i>Lua Library</i>	The Factorio Standard Library — unit-tested utilities for math, tables, strings, positions, and areas. More verbose than flib but extremely well-documented. Good for beginners learning the API. Optional if using flib.

Reference Mods (Study These)

The best way to learn Factorio modding is to read the source of polished mods that do similar things. These are the most relevant to The Reef:

Tool / Resource	Type	Purpose
Maraxisis	<i>Planet Mod</i>	The gold standard for Space Age planet mods. Adds a fully underwater planet with custom vehicles, structures, and mechanics. Study its planet prototype definition, surface property usage, and how it gates content behind a tech tree.
Cerys, Moon of Fulgora	<i>Space Location</i>	A space location (not a full-surface planet) that adds a nuclear scrap recycling loop. Directly analogous to The Reef's structure — study how it defines a non-landable destination and its asteroid/scrap mechanics.
Space Age base game	<i>Official Reference</i>	Wube publishes the full data for the base game. In your Factorio install, browse <code>data/space-age/prototypes/planet/</code> to see exactly how Vulcanus, Fulgora, Gleba, Aquilo, and the Shattered Planet are defined. This is the most authoritative reference you have.

Documentation & Community

Tool / Resource	Type	Purpose
lua-api.factorio.com	<i>API Docs</i>	The authoritative Factorio Lua API reference. Every prototype type, every runtime object, every event — all documented here. Bookmark it. Cross-reference with the FMTK type definitions for the best development experience.
wiki.factorio.com/Modding	<i>Wiki</i>	The Factorio Wiki modding portal. Covers the data lifecycle (<code>data.lua</code> → <code>data-updates.lua</code> → <code>data-final-fixes.lua</code>), prototype system, and has tutorials for beginners. Start here before diving into the API docs.
mods.factorio.com	<i>Mod Portal</i>	For finding reference mods and eventually publishing The Reef. Use the Lua Libraries category to discover additional utility mods.
The Foundry (foundrygg.com)	<i>Community Hub</i>	The community site for Factorio mod development. Links to PlanetsLib, a catalog of permissively-licensed graphics (useful for placeholder art), and a template mod to start

		from.
Factorio Discord #modding	<i>Community</i>	The official Factorio Discord has an active #modding channel. Experienced modders answer questions, and PlanetsLib maintainers are active there. Essential for tricky API questions.
Noise Tools (Earendel)	<i>Debug Mod</i>	A mod by the Space Age lead developer that visualizes noise expressions for world generation debugging. Install it when working on Phase 1's terrain/visual setup.

Graphics Resources

You'll need placeholder art early and final art before release. Options:

- Factorio Assets (via The Foundry) — A catalog of permissively-licensed graphics from the community that you can use directly in mods.
- Maraxis source — Released under a permissive license; its entity and item sprites are high quality and can serve as style references.
- PlanetsLib helper scripts — Includes a Python script (`generate_orbit_graphics.py`) for generating the orbit ring sprite that appears on the space map. Run it with: `uv run helper_scripts/generate_orbit_graphics.py [distance] [planet_name] [mod_name]`
- AI image generation — Acceptable for placeholder icons during development. Final release should use purpose-made pixel art at Factorio's standard resolutions (32x32, 64x64 icons; entity sprites at 32px/tile).

Plan of Work

Each phase builds on the last and is designed to teach specific modding concepts before they're needed. Start Phase 0 before anything else — it establishes the workflow you'll use for every subsequent step.

PHASE 0 Project Setup & Foundations

Skills: *Mod structure · info.json · data loading · testing workflow · Claude Code setup*

1

Install the Lua skill (mindrally/lua)

Install before anything else. Customize > Skills in the Claude desktop app, search 'lua' by mindrally, and enable it. This applies across all sessions including Claude Code.

2

Install and configure RTK

Install from Homebrew or GitHub (not crates.io — wrong package). Run `rtk gain` to verify, then `rtk init -g` to activate the Claude Code hook. Restart Claude Code. Check `rtk gain --history` after your first session to confirm savings.

3

Write the CLAUDE.md file

Create CLAUDE.md in the mod root before the first Claude Code session. Include: mod name and concept summary, current phase, Factorio 2.x / Space Age target version, key design decisions already made, and a note to always check lua-api.factorio.com for prototype field names rather than relying on training data. Also include a standing instruction for Claude Code to commit to git before starting any task and again after completing it successfully.

4

Create the mod skeleton

info.json, data.lua, control.lua stubs, settings.lua stub, locale/en/mod.cfg stub

5

Set up Git version control

Initialize a repo in the mod folder; commit the skeleton. Claude Code's checkpoint system also saves state before each edit, but git is your safety net for larger rollbacks. Set a standing policy in CLAUDE.md: commit before starting any task, and commit again after completing it successfully. This gives you a clean restore point for every unit of work.

6

Set up the testing workflow

Learn `/reload`, `log()` debugging, and the mod loading order (`data` → `data-updates` → `data-final-fixes` → `control`)

7

Register the mod with no content

Add a placeholder icon and confirm the mod appears in Factorio's mod list and loads without errors

8

Add flib and PlanetsLib as dependencies

Add to info.json dependencies array; confirm they load correctly alongside your mod

 **Tip:** Keep a test save specifically for mod development. Use `/cheat-mode` and `/editor` to spawn items and inspect entities freely. The `/reload` command reloads all mods without restarting Factorio.

PARALLEL WORKSTREAM Factorio Modding Skill

Runs alongside all phases. Build iteratively as errors are discovered and corrected.

P1 Create the initial Factorio modding skill

Start with a `SKILL.md` covering: the Factorio 2.x data lifecycle, common prototype fields and their gotchas, key API areas by phase, and version differences between 1.x and 2.x/Space Age that Claude commonly gets wrong. Scope: Factorio 2.x / Space Age only.

P2 Add a reference file: `prototype-cheatsheet.md`

Quick-lookup for the prototype types used in this mod: `space-location`, `asteroid`, `item`, `recipe`, `technology`, `entity`, `generator`, `thruster`. Key fields only — what is required, what is commonly wrong, and the correct 2.x field names.

P3 Add a reference file: `space-age-specifics.md`

Space platform APIs, asteroid spawning hooks, surface properties, `on_space_platform_*` events. This is the area most likely to be wrong in Claude training data since Space Age is relatively recent.

P4 Iterate: log and apply corrections

Every time Claude makes a Factorio-specific error during development — wrong field name, wrong event, 1.x API used instead of 2.x — log the correction in the skill. Over time the skill becomes a curated error-prevention system specific to this project.

P5 Package and install the final skill

Once the skill has been iterated through several phases, package it as a `.skill` file and install it via `Customize > Skills`. Other Factorio modders may find it useful — consider publishing it.

 **Tip:** The parallel workstream has no fixed schedule. Update the skill whenever you correct a Claude error. By Phase 3 or 4 you should have a skill that meaningfully reduces the error rate on Factorio API calls.

PHASE 1 The Destination

Skills: *Space-location prototypes · starmap UI · icons · technology gating*

6 Define The Reef as a space-location prototype

Model it on the `Shattered Planet` prototype (`data/space-age/prototypes/planet/`). Set `distance`, `orientation`, `magnitude`, and `solar_power_in_space`. No surface map — it's a non-landable destination like `Shattered Planet`.

7 Create a starmap icon

Use a placeholder for now. PlanetsLib's `generate_orbit_graphics.py` script will create the orbit ring graphic once you know the final distance/orientation values.

8

Place The Reef on the star map

Confirm it appears in the correct position relative to Fulgora and Aquilo. Adjust distance and orientation until the placement feels right narratively.

9

Create the discovery technology

A research that unlocks travel to The Reef. Gate it behind appropriate Space Age prerequisites (post-Fulgora recommended, given the scrap theme).

10

Define initial asteroid spawn definitions

Set up the approach corridor with a mix of vanilla asteroids that will later be replaced with scrap. Confirm platforms can travel to The Reef.

 **Tip:** The Shattered Planet in `data/space-age/prototypes/planet/shattered-planet.lua` is your closest template. It's a space-location (not a planet with a surface), which is exactly what The Reef needs.

PHASE 2 Asteroid → Scrap Pipeline

Skills: *Item prototypes · recipe prototypes · crusher/recycler integration · custom asteroids*

11

Create Starship Scrap items

Start with a single generic 'Starship Scrap' item. Optionally add variants later (Hull Plating Scrap, Conduit Scrap) for more interesting outputs.

12

Create scrap chunk asteroid entities

These replace/supplement vanilla asteroids in The Reef's approach corridor. Model on existing asteroid prototypes. They should look like jagged metal debris.

13

Update asteroid spawn definitions for The Reef

Gradually transition the spawn mix from vanilla rocks to scrap chunks as the platform gets closer to The Reef. The farther in, the more scrap and less rock.

14

Add crusher recipes: Scrap → raw resources

Generic Starship Scrap → iron, copper, steel, small chance of Dilithium Crystal. If using variants, give each a distinct output profile.

15

Add recycler recipes for scrap variants

Hook into the Fulgoran Recycler recipe system. Scrap variants → more refined or specialized outputs than the basic crusher yields.

16

Balance the scrap economy

Tune spawn rates and recipe outputs so the scrap pipeline is worth using but doesn't trivialize

the rest of the game. Compare yield-per-unit-time to Fulgoran scrap.

 **Tip:** Look at how Fulgora's scrap chunks are defined — they're the closest existing analog. The asteroid entity type is what gets collected by Asteroid Collectors on platforms.

PHASE 3 Dilithium & Tech Tree

Skills: Unique resources · fluid prototypes · technology trees · research dependencies

17 Create the Dilithium Crystal item

A rare drop from scrap processing. Consider whether it should also exist as a fluid (liquid Dilithium) for use in the generator — this opens up pipeline gameplay but adds complexity.

18 Design the full tech tree on paper first

Map out all The Reef technologies before implementing any. Identify what gates what, what the player bottlenecks are, and how long each tier should take. Share with Claude for feedback before implementation.

19 Implement the Dilithium tech chain

A linear chain of 4–6 technologies, each unlocking one or two structures from Phases 4–7. Gate the chain behind the Reef discovery tech from Phase 1.

20 Set correct technology dependencies

Ensure prerequisites reference appropriate vanilla Space Age techs (Fulgoran Recycling, Space Platform upgrades, etc.) so the player reaches The Reef at the right progression point.

21 Add Dilithium Science Pack (optional)

If desired, Dilithium can produce a unique science pack needed to research the later Reef technologies. This is the 'hardest' option but most in line with Space Age's design philosophy.

 **Tip:** Design the tech tree on paper (or in a spreadsheet) before coding it. It's much easier to restructure prerequisites before they're implemented than after. A rough rule: each tech should feel like a meaningful unlock, not just a tax.

PHASE 4 Structures: Power & Logistics

Skills: Entity prototypes · energy source definitions · inventory scripting · GUI

22 Dilithium Generator

A 'void' or burner-style energy source that produces steady power with no ongoing input (or minimal input like ice). Study how the fusion reactor defines its energy output. The 'set and forget' nature makes it ideal for platform startup.

23 Cargo Hatch (basic)

The trickiest structure in this phase. A single-item access point to the platform's central inventory. Requires scripting against LuaSurface inventory APIs and on_tick event handling. Keep v1 simple: one configured item, insert or extract only.

24 **Advanced Cargo Hatch**

Multi-item version of the Cargo Hatch. Place this deeper in the tech tree as a meaningful upgrade. The implementation extends the basic Cargo Hatch's script rather than replacing it.

25 **Cargo Hatch GUI**

Both Cargo Hatch types need a configuration UI for selecting which item(s) to pass through. Use flib's GUI module to build this — it handles much of the boilerplate for Factorio GUIs.

 **Tip:** The Cargo Hatch is your most novel mechanic. Consider building a minimal working version first (hardcoded item type, no GUI) to validate the inventory scripting approach before adding the configuration layer.

PHASE 5 Structures: Scrap & Defense

Skills: Spawner scripting · radius effects · shield entity behavior · on_tick events

26 **Electromagnetic Attractor**

Increases scrap chunk spawn rate in a wide radius and slowly pulls nearby chunks toward the attractor. Requires on_tick scripting to move existing entities and hooks into whatever asteroid spawn events Space Age exposes for platforms.

27 **Attractor visual effect**

A pulling beam or pulsing field effect to make the attractor's operation visible. Use the sprite/beam entity system — look at how existing beam weapons are defined for reference.

28 **Energy Shield Generator**

Provides a shared HP buffer to all structures within a medium radius. Scripted via on_entity_damaged events and shield pool management in script storage. Consumes significant power. Study how personal armor shields work for the HP-buffer logic.

29 **Shield Generator visual state**

Show active/inactive state via entity variation sprites or a rendered overlay. Players need to know when the shield is up and how much capacity remains (circuit output?).

 **Tip:** Check what events Space Age exposes for asteroid spawning on platforms before committing to the Attractor's design. If the event hook doesn't exist, you may need to poll on_tick instead — which works but is more resource-intensive.

PHASE 6 Structures: Transport

Skills: Cross-surface item transfer · fluid consumers · cooling cycles · circuit network

30

Cargo Railgun (planet-side)

Built on planets, fires a configured item type to a linked Railgun Catcher on a space platform. Requires Fluoroketone as fuel/coolant. Study how rocket silo item launches work for the cross-surface transfer model.

31

Railgun Catcher (platform-side)

Receives items launched by a Cargo Railgun. Needs a buffer inventory and a visual 'catching' animation. The pairing/linking mechanic (how does a Catcher know which Railgun is aimed at it?) needs design thought.

32

Fluoroketone cooling cycle

The Railgun consumes Fluoroketone and produces a 'used' coolant fluid that must be recycled or produced again. This creates the satisfying loop of: launch items → manage heat → recycle coolant → repeat.

33

Railgun/Catcher circuit integration

Allow circuit conditions to trigger launches and output signals for capacity/ready state. This makes the Railgun composable into larger automation schemes.

 **Tip:** The cross-surface item transfer is the most technically complex scripting in the entire mod. Research how mods like Space Exploration (1.x era) handled planet-to-ship transfers. The core pattern is: detect launch event → remove item from source inventory → teleport/create item in destination inventory.

PHASE 7 Ion Thruster

Skills: Thruster prototypes · circuit network behavior · fluid inputs · balancing

34

Basic Ion Thruster

Lower thrust than vanilla but shorter exhaust trail and only requires power + coolant. Study the thruster prototype definition in Space Age base data. Confirm what fluid inputs the prototype accepts natively.

35

Advanced Ion Thruster

Higher power, better coolant required, circuit-controllable output percentage. The circuit control (outputting at X% thrust) may require scripting if the prototype doesn't expose this natively — check the API first.

36

Thruster balance pass

Compare total thrust-to-power and thrust-to-fuel ratios against vanilla thrusters at all stages. The Advanced Ion Thruster should surpass vanilla thrusters only with significant power investment, not trivially.

37

Exhaust trail graphics

The 'shorter exhaust trail' is a visual differentiator. The thruster exhaust is defined in the prototype — shorten the particle emission zone and adjust particle lifetime to achieve the

tighter trail look.

 **Tip:** Implement the Basic Ion Thruster first and play-test it before building the Advanced version. Balance is much easier to feel in-game than to calculate on paper. Get a working thruster into a real platform run first.

PHASE 8 Polish & Release Prep

Skills: *Locale files · mod settings · graphics finalization · changelog · mod portal*

Write all locale strings

38 *Every item name, description, entity name, technology name, and tooltip needs a locale entry in locale/en/mod.cfg. Missing locale shows as __mod-name__item-name__ in game — easy to spot but tedious to fix at the end.*

Finalize icons and entity graphics

39 *Replace all placeholder art with final sprites. Factorio uses 32px/tile for entities, 32x32 or 64x64 for item icons. All sprites need a @2x high-res variant for modern displays.*

Add mod settings

40 *Allow players to toggle optional features (e.g. disable the Attractor's entity-pulling, adjust Dilithium drop rates). Use settings.lua and the startup/runtime-global/runtime-per-user setting tiers appropriately.*

Write the changelog

41 *Factorio has a specific changelog.txt format that the game parses and displays in the mod browser. Start writing it from Phase 0 — one entry per meaningful change is much easier than reconstructing history at the end.*

Create the mod portal thumbnail

42 *256x256 PNG. Should immediately communicate the mod's theme (space debris, alien tech). This is the first thing players see on the mod portal.*

Final balance playthrough

43 *Complete a full Space Age playthrough up to The Reef and through all its content. Time each phase of engagement. Adjust spawn rates, tech costs, and structure power budgets based on feel.*

Zip and upload to the mod portal

44 *The mod portal requires a .zip file. Use the zipmod.sh script pattern: `zip -r the-reef_1.0.0.zip the-reef/ --exclude '*.git*'`. Upload, write a description, and tag appropriately (Planets, New Content, Space Age).*

Design Notes & Open Questions

These are design decisions worth resolving before or during implementation. Flagged early because they affect architecture choices made in the first few phases.

Dilithium as Solid vs. Fluid

Dilithium Crystal (solid item) is simpler to implement and more approachable for players. Liquid Dilithium (fluid) opens up pipeline gameplay and the possibility of Dilithium-based fuel, but requires fluid entity definitions and adds complexity to the generator. Recommendation: start solid, add fluid variant in a post-release update if the community wants it.

Cargo Hatch Pairing Model

How does a Cargo Hatch know which platform inventory slot to access? Options: (A) It always accesses the platform's shared cargo hub directly — simplest, no pairing needed. (B) It pairs with a specific chest/buffer on the platform via a unique ID set in both — more flexible but requires a pairing UI. Option A is strongly recommended for v1.

Railgun Targeting

How does a Cargo Railgun on a planet know which platform to target? The platform needs to be 'in range' or 'in orbit' above the planet. Options: (A) Targets any platform currently above the planet — automatic but could cause confusion with multiple platforms. (B) Requires manual pairing like a linked chest. Needs design resolution before Phase 6.

Ion Thruster Circuit Control

The ability to set thrust output percentage via circuit network is a key differentiator for the Advanced Ion Thruster. Before implementing, verify whether the thruster prototype exposes a circuit input natively in Space Age 2.x. If not, this will require scripting with `on_tick` manipulation of the thruster's fluid consumption, which is more complex but achievable.

Progression Placement

The Reef is currently envisioned as post-Fulgora, pre-Aquilo. This makes thematic sense (scrap from alien ships, Dilithium as exotic power source) and ensures players already understand Recyclers and Crushers from Fulgora before arriving. Consider whether The Reef should also require visiting Aquilo — the energy shield and ion thruster tech could logically build on cryo and Aquilo's advanced tech.

The Electromagnetic Attractor's Pulling Mechanic

'Slowly drawing in scrap from a wide area' is evocative but complex. The implementation requires moving entity positions `on_tick`, which is expensive if many entities are being moved. Consider a simplified model: the Attractor increases spawn rate significantly within its radius (easy) but doesn't physically move existing chunks (the 'pull' is represented by spawn rate alone). This could be framed as 'the Attractor's field disrupts nearby debris orbits, causing them to drift toward it' — same fantasy, much simpler implementation.

Quick Reference: Key API Areas per Phase

Tool / Resource	Type	Purpose
Phase 1	<i>space-location prototype, technology prototype, asteroid_spawn_definition</i>	
Phase 2	<i>asteroid prototype, item prototype, recipe prototype, autoplace-control</i>	
Phase 3	<i>item prototype, fluid prototype, technology prototype, research-unit</i>	
Phase 4	<i>generator prototype (energy source), container prototype, script.on_event, LuaSurface inventory</i>	
Phase 5	<i>script.on_tick , script.on_entity_damaged, beam prototype, sprite-variation</i>	
Phase 6	<i>script remote calls, inventory manipulation, fluid-turret analogy for Railgun,</i>	

	<i>on_nth_tick</i>	
Phase 7	<i>thruster prototype, fluid-box definition, circuit- connector- definition</i>	
Phase 8	<i>locale format, mod-settings, changelog.txt format, zip packaging</i>	